1. Introduction to 8-Bit Microprocessors

2. Brief history of CP/M
    a. What is CP/M
    b. CP/M Commands
    c. CP/M Hardware
    d. CP/M Internals

3. Programming Languages: With examples
    a. Assembler - asm
    b. BASIC - mbasic
    c. Pascal - Turbo Pascal
    d. C - Aztec C
    e. FORTRAN
    f. COBOL

4. Modern CP/M Tools
    a. Modern Z-80 Hardware
    b. Emulators
    c. Utilities

5. Summary

Introduction to 8-Bit Microprocessors

8-bit microprocessors were the first widely used microprocessors in the computing industry, marking a major shift from mainframes and minicomputers to smaller, more affordable systems. The introduction of 8-bit processors in the 1970s enabled the production of personal computers, leading to the popularization of computing and setting the foundation for the modern computing landscape.

8008

The first commercial 8-bit processor was the Intel 8008 in 1972 which was originally intended for the Datapoint 2200 intelligent terminal. Intel's first 8-bit microprocessor, designed by a team including Ted Hoff, Stan Mazor, and Federico Faggin, who had also worked on the 4004.

The chip, limited by its 18-pin DIP, has a single 8-bit bus working triple duty to transfer 8 data bits, 14 address bits, and two status bits. The small package requires about 30 TTL support chips to interface to memory. For example, the 14-bit address, which can access "16 K × 8 bits of memory", needs to be latched by some of this logic into an external memory address register (MAR). The 8008 can access 8 input ports and 24 output ports.

For controller and CRT terminal use, this is an acceptable design, but it is rather cumbersome to use for most other tasks, at least compared to the next generations of microprocessors. A few early computer designs were based on it, but most would use the later and greatly improved Intel 8080 instead.

8080

This was followed by the 8080 in 1974, paving the way for the development of personal computers and the modern computing landscape. It was an improved version of the 8008, with a 40-pin package and a clock speed of 2 MHz, it was crucial for early personal computers like the Altair 8800. Most competitors to Intel started off with such character oriented 8-bit

microprocessors. Modernized variants of these 8-bit machines are still one of the most common types of processor in embedded systems.

6800

The Motorola 6800, also introduced in 1974, was another influential design, showcasing innovation and contributing to the democratization of computing. Built from scratch, unlike the 8008 and 8080, it took a different approach with fewer registers and a more regular instruction set. It was used in "grown-up" devices like cash machines. This was used in the SWTPC 6800 Computer System, or SWTPC 6800, an early microcomputer developed by the Southwest Technical Products Corporation and introduced in 1975. The SWTPC 6800 was one of the first microcomputers based around the Motorola 6800.

6502

The MOS Technology 6502 was released in 1975. When it was introduced, the 6502 was the least expensive microprocessor on the market by a considerable margin. It initially sold for less than one-sixth the cost of competing designs from larger companies, such as the 6800 or Intel 8080. The 6502 and variants of it, were used in personal computers, such as the Apple I, Apple II, Atari 8-bit computers, BBC Micro, PET, VIC-20, and in home video game consoles such as the Atari 2600 and the Nintendo Entertainment System.

Z-80

The Zilog Z-80 released in 1976, was developed by Zilog as an enhanced version of the 8080, offering improvements in interrupt handling, instruction set, and memory management. The Z-80 was designed to be software-compatible with the Intel 8080, offering a compelling alternative due to its better integration and increased performance. Along with the 8080's seven registers and flags register, the Z80 introduced an alternate register set, two 16-bit index registers, and additional instructions, including bit manipulation and block copy/search.

Zilog was founded in 1974 by Federico Faggin and Ralph Ungermann, who were soon joined by Masatoshi Shima. All three had left Intel after working on the 4004 and 8080 microprocessors.

Other Chips

The Intel 8085, released in 1976, was an enhanced version of the 8080, aimed at systems requiring fewer chips.

There are other 8-bit microprocessors including offerings by: Fairchild, Microchip, RCA, and Signetics, but we will not be covering them here today.

Each of the processor families had its own instruction set and assembly language. The Z-80 was compatible with the 8080 in that it could run the native 8080 code.

Going forward we are only going to be looking at the 8080, and the Z-80, as this is what CP/M was developed to run on in the beginning.

Brief History of CP/M

CP/M, originally standing for Control Program/Monitor and later Control Program for Microcomputers, is a mass-market operating system created in 1974 for Intel 8080/85-based microcomputers by Gary Kildall of Digital Research, Inc. CP/M is a disk operating system and its purpose is to organize files on a magnetic storage medium, and to load and run programs stored on a disk. Initially confined to single-tasking on 8-bit processors and no more than 64 kilobytes of memory, later versions of CP/M added multi-user variations and were migrated to 16-bit processors.

Gary Kildall originally developed CP/M as an operating system to run on an Intel Intellec-8 development system, equipped with a Shugart Associates 8-inch floppy-disk drive interfaced via a custom floppy-disk controller. It was written in Kildall's own PL/M (Programming Language for Microcomputers). Various aspects of CP/M were influenced by the TOPS-10 operating system of the DECsystem-10 mainframe computer, which Kildall had used as a development environment.

The CP/M name follows a prevailing naming scheme of the time, as in Kildall's PL/M language, and Prime Computer's PL/P (Programming Language for Prime), both suggesting IBM's PL/I; and IBM's CP/CMS operating system, which Kildall had used when working at the Naval Postgraduate School.

CP/M supported a wide range of computers based on the 8080 and Z80 CPUs. An early outside licensee of CP/M was Gnat Computers, an early microcomputer developer out of San Diego, California. In 1977, the company was granted the license to use CP/M 1.0 for any micro they desired for $90. Within the year, demand for CP/M was so high that Digital Research was able to increase the license to tens of thousands of dollars.

CP/M 2.0

Under Kildall's direction, the development of CP/M 2.0 was mostly carried out by John Pierce in 1978. Kathryn Strutynski, a friend of Kildall from Naval Postgraduate School (NPS), became the fourth employee of Digital Research Inc. in early 1979. She started by debugging CP/M 2.0, and later became influential as key developer for CP/M 2.2 and CP/M Plus. Other early developers of the CP/M base included Robert "Bob" Silberstein and David "Dave" K. Brown.

CP/M eventually became the de facto standard and the dominant operating system for microcomputers, in combination with the S-100 bus computers. This computer platform was widely used in business through the late 1970s and into the mid-1980s.

The operating system was described as a "software bus", allowing multiple programs to interact with different hardware in a standardized way. Programs written for CP/M were typically portable among different machines, usually requiring only the specification of the escape sequences for control of the screen and printer. This portability made CP/M popular, and much more software was written for CP/M than for operating systems that ran on only one brand of hardware.

One restriction on portability was that certain programs used the extended instruction set of the Z80 processor and would not operate on an 8080 or 8085 processor. Another was graphics routines, especially in games and graphics programs, which were generally machine-specific as they used direct hardware access for speed, bypassing the OS and BIOS (this was also a common problem in early DOS machines).

CP/M increased the market size for both hardware and software by greatly reducing the amount of programming required to port an application to a new manufacturer's computer. An important driver of software innovation was the advent of (comparatively) low-cost microcomputers running CP/M,

as independent programmers and hackers bought them and shared their creations in user groups.

MP/M

MP/M (Multi-Programming Monitor Control Program) is a multi-user version of the CP/M operating system, created by Digital Research developer Tom Rolander in 1979.It allowed multiple users to connect to a single computer, each using a separate terminal.

MP/M was a fairly advanced operating system for its era, at least on microcomputers. It included a priority-scheduled multitasking kernel (before such a name was used, the kernel was referred to as the nucleus) with memory protection, concurrent input/output (XIOS) and support for spooling and queuing. It also allowed for each user to run multiple programs, and switch between them.

The 8-bit system required a 8080 (or Z80) CPU and a minimum of 32 KB of RAM to run, but this left little memory for user applications. In order to support reasonable setups, MP/M allowed for memory to be switched in and out of the machine's "real memory" area. So for instance a program might be loaded into a "bank" of RAM that was not addressable by the CPU, and when it was time for the program to run that bank of RAM would be "switched" to appear in low memory (typically the lower 32 or 48 KB) and thus become visible to the OS. This technique, known as bank switching, was subsequently added to the single user version of CP/M with version 3.0.

MP/M II 2.0 added file sharing capabilities in 1981, MP/M II 2.1 came with extended file locking in January 1982.

CP/M 3

The last 8-bit version of CP/M was version 3, often called CP/M Plus, released in 1983. Its BDOS was designed by David K. Brown. It

incorporated the bank switching memory management of MP/M in a single-user single-task operating system compatible with CP/M 2.2 applications. CP/M 3 could therefore use more than 64 KB of memory on an 8080 or Z80 processor. The system could be configured to support date stamping of files. The operating system distribution software also included a relocating assembler and linker. CP/M 3 was available for the last generation of 8-bit computers, notably the Amstrad PCW, the Amstrad CPC, the ZX Spectrum +3, the Commodore 128, MSX machines and the Radio Shack TRS-80 Model 4.

16 Bit Versions

There were versions of CP/M for some 16-bit CPUs as well.

The first version in the 16-bit family was CP/M-86 for the Intel 8086 in November 1981. Kathryn Strutynski was the project manager for the evolving CP/M-86 line of operating systems. At this point, the original 8-bit CP/M became known by the retronym CP/M-80 to avoid confusion.

Soon following CP/M-86, another 16-bit version of CP/M was CP/M-68K for the Motorola 68000. The original version of CP/M-68K in 1982 was written in Pascal/MT+68k, but it was ported to C later on. CP/M-68K, already running on the Motorola EXORmacs systems, was initially to be used in the Atari ST computer, but Atari decided to go with a newer disk operating system called GEMDOS. CP/M-68K was also used on the SORD M68 and M68MX computers.

In 1982, there was also a port from CP/M-68K to the 16-bit Zilog Z8000 for the Olivetti M20, written in C, named CP/M-8000.

These 16-bit versions of CP/M required application programs to be re-compiled for the new CPUs. Some programs written in assembly language could be automatically translated for a new processor. One tool for this was Digital Research's XLT86, which translated .ASM source code for the Intel 8080 processor into .A86 source code for the Intel 8086. The translator

would also optimize the output for code size and take care of calling conventions, so that CP/M-80 and MP/M-80 programs could be ported to the CP/M-86 and MP/M-86 platforms automatically. XLT86 itself was written in PL/I-80 and was available for CP/M-80 platforms as well as for VAX/VMS.

Later versions of CP/M-86 made significant strides in performance and usability and were made compatible with MS-DOS. To reflect this compatibility the name was changed, and CP/M-86 became DOS Plus, which in turn became DR-DOS.

Summary

In the early 1980s, Digital Research had sold more than 250,000 CP/M licenses; InfoWorld stated that the actual market was likely larger because of sub-licenses. Many different companies produced CP/M-based computers for many different markets; the magazine stated that "CP/M is well on its way to establishing itself as the small-computer operating system". Even companies with proprietary operating systems, such as Heath/Zenith (HDOS), offered CP/M as an alternative for their 8080/Z80-based systems. Also, an estimated 2000 CP/M applications existed. Many expected that it would be the standard operating system for 16-bit computers.

In 1980 IBM approached Digital Research, at Bill Gates' suggestion, to license a forthcoming version of CP/M for its new product, the IBM Personal Computer. Upon the failure to obtain a signed non-disclosure agreement, the talks failed, and IBM instead contracted with Microsoft to provide an operating system.

Many of the basic concepts and mechanisms of early versions of MS-DOS resemble those of CP/M. Internals like file-handling data structures are identical, and both refer to disk drives with a letter (A:, B:, etc.). MS-DOS's main innovation was its FAT file system. This similarity made it easier to port popular CP/M software like WordStar and dBase. However, CP/M's

concept of separate user areas for files on the same disk was never ported to MS-DOS. Since MS-DOS has access to more memory (as few IBM PCs were sold with less than 64 KB of memory, while CP/M can run in 16 KB if necessary), more commands are built into the command-line shell, making MS-DOS somewhat faster and easier to use on floppy-based computers.

BYTE in 1982 described MS-DOS and CP/M as David and Goliath, the magazine stated that MS-DOS was "much more user-friendly, faster, with many more advantages, and fewer disadvantages". InfoWorld stated in 1984 that efforts to introduce CP/M to the home market had been largely unsuccessful and most CP/M software was too expensive for home users. In 1986 the magazine stated that Kaypro had stopped production of 8-bit CP/M-based models to concentrate on sales of MS-DOS compatible systems, long after most other vendors had ceased production of new equipment and software for CP/M.

CP/M rapidly lost market share as the micro-computing market moved to the IBM-compatible platform, and never regained its former popularity. Byte magazine, one of the leading industry magazines for microcomputers, essentially ceased covering CP/M products within a few years of the introduction of the IBM PC. For example, in 1983 there were still a few advertisements for S-100 boards and articles on CP/M software, but by 1987 these were no longer found in the magazine.

What is CP/M?

CP/M is a monitor control program for microcomputer system development that uses floppy disks or Winchester hard disks for backup storage. Using a computer system based on an 8080 or Z-80 compatible microcomputer, CP/M provides an environment for program construction, storage, and editing, along with assembly and program checkout facilities. CP/M can be easily altered to execute with any computer configuration that uses a compatible Central Processing Unit (CPU) and has at least 20K bytes of main memory with up to 16 disk drives. Although the standard Digital Research version operates on a single-density Intel MDS 800, several different hardware manufacturers support their own input-output (I/O) drivers for CP/M.

The CP/M monitor provides rapid access to programs through a comprehensive file management package. The file subsystem supports a named file structure, allowing dynamic allocation of file space as well as sequential and random file access. Using this file system, a large number of programs can be stored in both source and machine executable form.

CP/M 2 is a high-performance, single console operating system that uses table-driven techniques to allow field reconfiguration to match a wide variety of disk capacities. All fundamental file restrictions are removed, maintaining upward compatibility from previous versions of release 1.

Features of CP/M 2 include field specification of one to sixteen logical drives, each containing up to eight megabytes. Any particular file can reach the full drive size with the capability of expanding to thirty-two megabytes in future releases. The directory size can be field-configured to contain any reasonable number of entries, and each file is optionally tagged with Read-Only and system attributes. Users of CP/M 2 are physically separated by user numbers, with facilities for file copy operations from one user area to another. Powerful relative-record random access functions are present in

CP/M 2 that provide direct access to any of the 65536 records of an eight-megabyte file.

CP/M also supports ED, a powerful context editor, ASM, an Intel-compatible assembler, and DDT, debugger subsystems. Optional software includes a powerful Intel-compatible macro assembler, symbolic debugger, along with various high-level languages. When coupled with CP/M's Console Command Processor (CCP), the resulting facilities equal or exceed similar large computer facilities.

CP/M is logically divided into several distinct parts in memory:

- BIOS (Basic I/O System), hardware-dependent
- BDOS (Basic Disk Operating System)
- CCP (Console Command Processor)
- TPA (Transient Program Area)

BIOS

The BIOS provides the primitive operations necessary to access the disk drives and to interface standard peripherals: teletype, CRT, paper tape reader/punch, and user-defined peripherals. You can tailor peripherals for any particular hardware environment by patching this portion of CP/M. If a computer manufacturer wanted to be able to run CP/M on their hardware, they would have to create a version of the BIOS that understood the hardware. This was the beginning of the concept of the Hardware Abstraction Layer that would allow CM/P to run on so many different hardware platforms.

BDOS

The BDOS contains all primitive functions (available to CP/M and user programs) to deal with disk drives, keyboard and screen, etc. It provides disk management by controlling one or more disk drives containing independent file directories. The BDOS implements disk allocation

strategies that provide fully dynamic file construction while minimizing head movement across the disk during access. The BDOS has entry points that include the following primitive operations, which the program accesses:

- SEARCH looks for a particular disk file by name.
- OPEN opens a file for further operations.
- CLOSE closes a file after processing.
- RENAME changes the name of a particular file.
- READ reads a record from a particular file.
- WRITE writes a record to a particular file.
- SELECT selects a particular disk drive for further operations.

CCP

The CCP provides a symbolic interface between your console and the remainder of the CP/M system. The CCP reads the console device and processes commands, which include listing the file directory, printing the contents of files, and controlling the operation of transient programs, such as assemblers, editors, and debuggers.

TPA

The last segment of CP/M memory is the area called the Transient Program Area (TPA). The TPA holds programs that are loaded from the disk under command of the CCP. During program editing, for example, the TPA holds the CP/M text editor machine code and data areas. Similarly, programs created under CP/M can be checked out by loading and executing these programs in the TPA.

Any or all of the CP/M component subsystems can be overlaid by an executing program. That is, once a user's program is loaded into the TPA, the CCP, BDOS, and BIOS areas can be used as the program's data area. A bootstrap loader is programmatically accessible whenever the BIOS portion is not overlaid; thus, the user program need only branch to the bootstrap loader at the end of execution and the complete CP/M monitor is reloaded from disk.

CP/M Commands

CP/M has two types of commands: built-in and transient. If the CCP does not recognize the command as one of the built-in commands, it then searches the current disk for a file of the same name as the command and tries to load and execute it. In this way CP/M can be extended by any user developed program as a command.

The built in commands are part of the CCP and are always available from the command prompt

- **ERA** - Erased one or more files from the disk
- **DIR** - Displays a directory of filenames on the disk
- **REN** - Renames a specified file
- **SAVE** - Saves blocks of memory contents to a file
- TYPE - Displays the contents of a file on the console
- USER - Changes the user number for disk access

The transient commands are loaded into the TPA from disk and executed. Here are some of the typical transient commands bundled with CP/M

- **STAT** - Lists the number of bytes of storage remaining on the currently logged disk, provides statistical information about particular files, and displays or alters device assignment.
- **ASM** - Loads the CP/M assembler and assembles the specified program from disk, generating an Intel HEX machine code format.
- **LOAD** - Loads the file in Intel HEX machine code format and produces a file in machine executable form which can be loaded into the TPA. This loaded program becomes a new command under the CCP.
- **DDT** - Loads the CP/M debugger into TPA and starts execution.
- PIP - Loads the Peripheral Interchange Program for subsequent disk file and peripheral transfer operations.
- **ED** - Loads and executes the CP/M text editor program.
- **SYSGEN** - Creates a new CP/M system disk.
- **SUBMIT** - Submits a file of commands for batch processing.
- **DUMP** - Dumps the contents of a file in hex.

- **MOVCPM** - Regenerates the CP/M system for a particular memory size.

CP/M Hardware

A minimal 8-bit CP/M system would contain the following components:

- A computer terminal using the ASCII character set
- An Intel 8080 (and later the 8085) or Zilog Z80 microprocessor
- The NEC V20 and V30 processors support an 8080-emulation mode that can run 8-bit CP/M on a PC-DOS/MS-DOS computer so equipped, though any PC clone could run CP/M-86.
- At least 16 kilobytes of RAM, beginning at address 0. The CPU could support up to 64 kilobytes.
- A means to bootstrap the first sector of the diskette.
- At least one floppy-disk drive

The only hardware system that CP/M, as sold by Digital Research, would support was the Intel 8080 Development System. Manufacturers of CP/M-compatible systems customized portions of the operating system for their own combination of installed memory, disk drives, and console devices.

CP/M would also run on systems based on the Zilog Z80 processor since the Z80 was compatible with 8080 code. While the Digital Research distributed core of CP/M (BDOS, CCP, core transient commands) did not use any of the Z80-specific instructions, many Z80-based systems used Z80 code in the system-specific BIOS, and many applications were dedicated to Z80-based CP/M machines.

Digital Research subsequently partnered with Zilog and American Microsystems to produce Personal CP/M, a ROM-based version of the operating system aimed at lower-cost systems that could potentially be equipped without disk drives. First featured in the Sharp MZ-800, a cassette-based system with optional disk drives, Personal CP/M was described as having been "rewritten to take advantage of the enhanced Z-80 instruction set" as opposed to preserving portability with the 8080. American Microsystems announced a Z80-compatible microprocessor, the S83, featuring 8 KB of in-package ROM for the operating system and BIOS, together with comprehensive logic for interfacing with 64-kilobit

dynamic RAM devices. Unit pricing of the S83 was quoted as $32 in 1,000 unit quantities.

On most machines the bootstrap was a minimal boot loader in ROM combined with some means of minimal bank switching or a means of injecting code on the bus (since the 8080 needs to see boot code at Address 0 for start-up, while CP/M needs RAM there); for others, this bootstrap had to be entered into memory using front-panel controls each time the system was started.

CP/M used the 7-bit ASCII set. The other 128 characters made possible by the 8-bit byte were not standardized. For example, one Kaypro used them for Greek characters, and Osborne machines used the 8th bit set to indicate an underlined character. WordStar used the 8th bit as an end-of-word marker. International CP/M systems most commonly used the ISO 646 norm for localized character sets, replacing certain ASCII characters with localized characters rather than adding them beyond the 7-bit boundary.

CP/M Internals

The reason most people are still drawn to CP/M is because it is so easy to fully understand the system, up from the tiniest detail. Yet, CP/M is the direct predecessor of MS-DOS (which was modeled very closely after CP/M) and has full functionality for normal use. If you know CP/M, you understand the low-level basics of any PC, and it gives you a level of understanding of the hardware that you'd never gain with, for instance, Linux. In short, understanding CP/M is relatively easy, and it gives you an insight in today's computers that is hard to obtain in any other way.

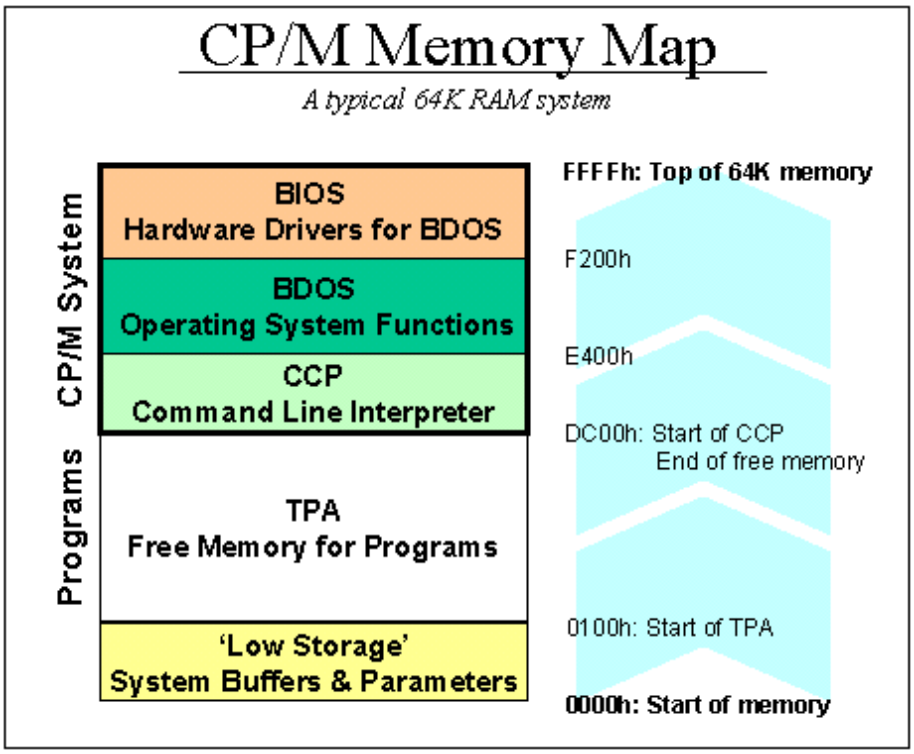
CP/M essentially provides programs with a set of function calls that allow them to communicate with the computer's I/O devices in a standardized manner. These system calls (the BDOS functions) ensure that user programs never have to bother with how the computer hardware stores a file or puts text on a screen - instead, the system calls can be relied upon to do the job. That concept is the basis for any operating system, and it enables computers that are very different in hardware to run the same programs. Here is a list with the CP/M BDOS calls - the operating system services.

The second service that CP/M provides is a command line interface and a number of small support programs that enable the user to maintain his computer and files. For CP/M, the list of support programs is minimal but complete: programs to edit files (ED, never to be used as it is truly bad. Instead, use VDE or WordStar) and copy (PIP) files, create assembler (ASM and LOAD) programs and debug (DDT) them.

Memory Map

CP/M requires a minimum of 20K RAM, although realistically, 48K is the bare minimum. Most systems have a maximum of 64K. The chart below shows the memory map of a CP/M system: the first 256 bytes (100 hexadecimal) are used as a scratchpad by the operating system. It

contains buffers, parameters and other transient data. This area is called Low Storage.



Moving up, the area where the user loads and runs his programs is called the TPA, Transient Program Area. The size of this area depends on the amount of RAM - its size is whatever is left after CP/M has taken what it needs. For a typical 64K system, the TPA stops at DC00h, but this number can vary.

Running a program from the command line does nothing more that load the file into memory, starting at address 0100h, and start executing the code at 0100h after loading.

The top segment of memory is used by CP/M. First, the CCP area contains the command line user interface. Then, the BDOS contains all primitive functions (available to CP/M and user programs) to deal with disk drives, keyboard and screen, etc. BDOS and CCP are the same across CP/M machines, they do not need any customization - other than the need for them to be relocated (shifted up or down in memory) if the size of total

memory changes after a user forked out to buy another 16K of RAM, for instance.

Lastly, the BIOS sits at the top end of memory. It normally is about 600h bytes in size, but can be larger if there is a lot of special hardware that needs to be managed. The BIOS is a function library that supports BDOS: anything that is hardware-specific needs to be done in the BIOS functions. BDOS, for instance, has a function for putting text on the screen, but will hand over the physical work of flipping bits in chips to the BIOS. BDOS has no idea whether the screen is a serial terminal or a graphics board.

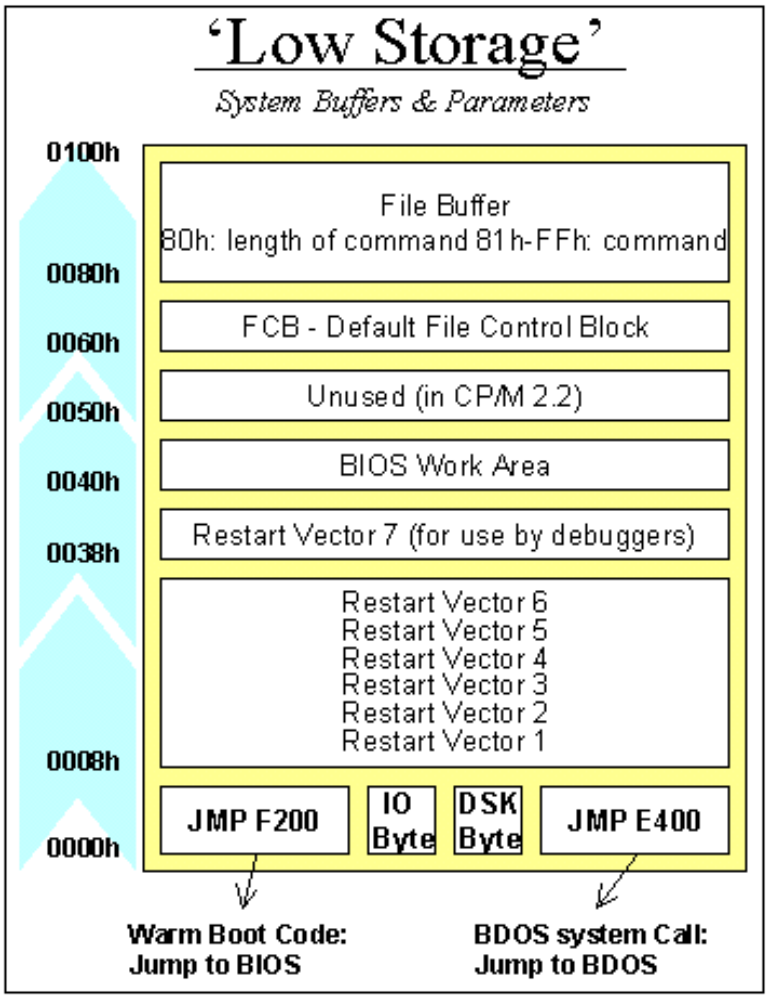
The Boot Process

To start up a CP/M computer, some boot-up code needs to put the CP/M code into memory, and all hardware must be initialized properly. The Z80 is hardwired to start executing program code at location 0000h when it is switched on. There are various schemes to ensure that proper startup code is stored there; but the most common method is to have a ROM at location 0000h that does all it needs to do, and then switch that ROM out for RAM memory after the initial start-up has been completed.

Usually, the ROM will load the CP/M boot sector from disk, store it in memory from DC00h on-wards (in a typical 64K system), and start executing the BIOS code. CP/M is then in charge and the Low Storage area is properly set up.

The Low Storage Area

The first 256 bytes of memory play a crucial role in CP/M. The area is initialized by the BIOS during the boot process, and then handed over to the CCP for maintenance. User programs will use specific parts of Low Storage when they need to access disk drives, or change I/O assignments (using the IO Byte).



Low Storage - The First Eight Bytes of Memory

Starting at the bottom of the memory map, the first three bytes of code contain the machine language instruction JMP F200h (or whatever address is the start of the BIOS). Because the Z80 processor starts itself up by beginning to execute whatever it finds at memory location 00, these three bytes are a logical place for the system reset routine. Executing the code causes the BIOS to reinitialize the Low Storage Area and forget about anything.

The following byte is the IO Byte. In good CP/M implementations (many computer vendors were sloppy in using this functionality), you can re-assign the real devices to CP/M virtual devices. For instance, you can

assign the keyboard hardware to what CP/M sees as the serial port. This byte serves as a switch between real and virtual devices and is a GREAT idea.

Byte 04h logs the current default drive (in bits 0-3, 0 being A: and 1 being B:, etc) and the current user (in bits 4-7). The user code is normally 0, but through the USER x command, it can be changed.

Bytes 05h-07h contain the machine language instruction JMP E400 (i.e., Jump to BDOS). Calling address 05h is the standard way in which programs use the services of CP/M.

Low Storage - the middle part

Moving further up in the memory map, the restart vectors starting at byte 08h can be initialized by the BIOS or a user program. Each interrupt level of the Z80 has a vector here. If an interrupt occurs, the Z80 will stop with whatever it is doing, and jump execution to one of these vectors depending on the level of the interrupt (0 to 6). Interrupt 7 is reserved for a debugger. If a debugger like DDT is loaded, it'll put the instruction JMP XXXX in bytes 38h-3Ah, where XXXX is the address where the debugger resides. Many earlier computers had a debug button, which when pressed would simply create a hardware interrupt 7 signal to land the user in the debugger. CP/M doesn't use interrupts itself, so these vectors can be used for whatever interrupt-generating hardware is built into the system.

The area from 40h upwards is used as a variable scratchpad by the BIOS for normally undocumented purposes. The area above, starting at 50h, is unused by CP/M 2.2 (not true for MP/M). It is a great place to store small programs that can remain resident in the computer.

Low Storage - the upper part

The File Control Block resides in bytes 60h-80h. This is a group of parameters that needs to be filled in before a program calls a BDOS

function that has anything to do with disk drives. In the FCB, the program stores the drive it wants to look at, its file name, and any other relevant details. The FCB is explained further down in this text.

Above 80h is where the CCP puts the DMA buffer. It also uses that memory for the arguments that are parsed for each command as it is executed. This is called the command tail. The first byte is the number of characters, without the final carriage return, followed by the upper-case representation of all the arguments

BDOS Calls

All of the services of CP/M are accessed through the BDOS. For instance, if a program wants to print the character 'A' to the screen:

```
MVI     A,65
MVI     C,02H
CALL    0005h
```

- it loads the value 65 (for A) in the Z80's accumulator,
- loads the value 02h (for BDOS service #, print character) in the Z80's C register
- and simply does a CALL 0005 to execute the BDOS request
- CP/M will put the character on the screen, and then return execution to the program.

CP/M facilities that are available for access by transient programs fall into two general categories: simple device I/O and disk file I/O.

The simple device operations are:

- read a console character
- write a console character
- read a sequential character
- write a sequential character
- get or set I/O status

- print console buffer
- interrogate console ready

The following FDOS operations perform disk I/O:

- disk system reset
- drive selection
- file creation
- file close
- directory search
- file delete
- file rename
- random or sequential read
- random or sequential write
- interrogate available disks
- interrogate selected disk
- set DMA address
- set/reset file indicators.

As mentioned above, access to the BDOS functions is accomplished by passing a function number and information address through the primary point at location BOOT+0005H.

In general, the function number is passed in register C with the information address in the double byte pair DE. Single byte values are returned in register A, with double byte values returned in HL, a zero value is returned when the function number is out of range. For reasons of compatibility, register A = L and register B = H upon return in all cases. Note that the register passing conventions of CP/M agree with those of the Intel PL/M systems programming language.

CP/M functions and their numbers are listed below.

Number	**Function**	**Number**	**Function**
0	System Reset	19	Delete File

1	Console Input	20	Read Sequential
2	Console Output	21	Write Sequential
3	Reader Input	22	Make File
4	Punch Output	23	Rename File
5	List Output	24	Return Login Vector
6	Direct Console I/O	25	Return Current Disk
7	Get I/O Byte	26	Set DMA Address
8	Set I/O Byte	27	Get Addr(Alloc)
9	Print String	28	Write Protect Disk
10	Read Console Buffer	29	Get R/O Vector
11	Get Console Status	30	Set File Attributes
12	Return Version Number	31	Get Addr(Disk Parms)
13	Reset Disk System	32	Set/Get User Code
14	Select Disk	33	Read Random
15	Open File	34	Write Random
16	Close File	35	Compute File Size
17	Search for First	36	Set Random Record
18	Search for Next	37	Reset Drive
		40	Write Random with Zero Fill

Programming Languages for CP/M

There are many programming languages for CP/M.

The first, PL/M, an acronym for Programming Language for Microcomputers, is a high-level language conceived and developed by Gary Kildall in 1973 for Hank Smith at Intel for the Intel 8008. It was later expanded for the newer Intel 8080.

The 8080 had enough power to run the PL/M compiler, but lacked a suitable form of mass storage. In an effort to port the language from the PDP-10 to the 8080, Kildall used PL/M to write a disk operating system that allowed a floppy disk to be used. This was the basis of CP/M.

Assembler has always been the first available programming language for a new processor. With the 8080, people were hand assembling their code and just toggling in the op–codes to get their programs to run. An assembler and linker / loader were supplied with CP/M.

BASIC gained popularity, particularly for general-purpose programming and educational purposes. CP/M came with the Microsoft Basic interpreter MBASIC.

Pascal, known for its structured programming approach, also gained traction on CP/M, providing an alternative to BASIC and assembly for more complex applications.

While other languages like FORTRAN and COBOL were prevalent during the same period, they were more common on larger mainframe systems and not as widely used on CP/M.

Assembly language remained crucial for close-to-hardware tasks, while BASIC and Pascal offered more user-friendly programming environments for general applications.

It would be a large undertaking to discuss all of the languages for CP/M development. The ones I am going to cover here are:

- Assembler
- BASIC
- Pascal
- C

I will give a brief history, mostly pulled from Wikipedia, follow up with some personal anecdotes and then conclude each language with a simple Hello World program.

Additionally there are some other languages that have been archived and made available for native development:

- Fortran-80
- COBOL-80
- PL/M
- Turbo Modula
- AP/L 2
- Forth-80
- Lisp 80
- SLR

I have used a few of these languages in different environments and can discuss them if there is any interest.

Assembly Languages

In computer programming, assembly language (alternatively assembler language or symbolic machine code), often referred to simply as assembly and commonly abbreviated as ASM or asm, is any low-level programming language with a very strong correspondence between the instructions in the language and the architecture's machine code instructions. Assembly language usually has one statement per machine instruction (1:1), but constants, comments, assembler directives, symbolic labels of, e.g., memory locations, registers, and macros are generally also supported.

Because assembly depends on the machine code instructions, each assembly language is specific to a particular computer architecture.

Sometimes there is more than one assembler for the same architecture, and sometimes an assembler is specific to an operating system or to a particular family of operating systems. Most assembly languages do not provide specific syntax for operating system calls, and most assembly languages can be used universally with any operating system, as the language provides access to all the real capabilities of the processor, upon which all system call mechanisms ultimately rest. In contrast to assembly languages, most high-level programming languages are generally portable across multiple architectures but require interpreting or compiling, much more complicated tasks than assembling.

Today, it is typical to use small amounts of assembly language code within larger systems implemented in a higher-level language, for performance reasons or to interact directly with hardware in ways unsupported by the higher-level language. For instance, just under 2% of version 4.9 of the Linux kernel source code is written in assembly; more than 97% is written in C.

For me the first exposure to Assembler was the big boy IBM 360 Assembler in my first university computer 101 class, An Introduction to

Computer Programming. A summer course of 9 weeks. Split up into 3 sections, IBM Assembler, Fortran and Basic. The first two parts were done with Punched cards, a card reader and line printer connected to a remote IBM 360 system at another Research Center across the state. The third section was BASIC-Plus on the Universities PDP-11/40 mentioned earlier.

I would later dig into DEC Macro-11 on the PDP, then when the Apple][s came in 6502 assembler was a fun study. I wrote an audio digitizer routine that used the Apple's Joystick port to record about 10 seconds of audio into about 40K of RAM, and then play it back over the tiny speaker. Needless to say is sounded terrible.

Professionally I first used assembly on a Kaypro 10. This allowed me to write several file utilities for cleaning up "locked" RM-COBOL data files that were left locked by the applications when they crashed. Which was often. The company I worked for gave them to customers to help their productivity. Later I would develop some home automation tools in 8085 Assembler and then I did some embedded systems development for a PIC micro-controller for refrigeration control.

Sample Code

The assembler hello world program is a simple single call to BDOS to print the message string:

```
C>type hello.asm
        ORG     100H

BDOS    EQU     0005H                   ; LOCATION OF BDOS ENTRY POINT
BOOT    EQU     0000H                   ; LOCATION OF BOOT REQUEST

START:
        MVI     C,9                     ; BDOS REQUEST 9 - PRINT STRING
        LXI     D,MESSAGE               ; OUR STRING TO PRINT
        CALL    BDOS
        JMP     BOOT                    ; EXIT TO CP/M

MESSAGE:
        DB      13,10,'Hello World from CP/M!',13,10,'$'
```

```
        END     START

C>asm hello
CP/M ASSEMBLER - VER 2.0
0126
000H USE FACTOR
END OF ASSEMBLY

A>load hello

FIRST ADDRESS 0100
LAST  ADDRESS 0125
BYTES READ    0026
RECORDS WRITTEN 01


C>hello

Hello World from CP/M!

C>
```

BASIC

Beginners All-purpose Symbolic Instruction Code

BASIC is a family of general-purpose, high-level programming languages designed for ease of use. The original version was created by John G. Kemeny and Thomas E. Kurtz at Dartmouth College in 1963. They wanted to enable students in non-scientific fields to use computers. At the time, nearly all computers required writing custom software, which only scientists and mathematicians tended to learn.

The first microcomputer version of BASIC was co-written by Bill Gates, Paul Allen and Monte Davidoff for their newly formed company, Micro-Soft. This was released by MITS in punch tape format for the Altair 8800 shortly after the machine itself, immediately cementing BASIC as the primary language of early microcomputers. Members of the Homebrew Computer Club began circulating copies of the program, causing Gates to write his Open Letter to Hobbyists, complaining about this early example of software piracy.

Microsoft sold a CP/M BASIC compiler (known as BASCOM) which used a similar source language to MBASIC. A program debugged under MBASIC could be compiled with BASCOM. Since program text was no longer in memory and the run-time elements of the compiler were smaller than the interpreter, more memory was available for user data. Speed of real program execution increased about 3 fold.

Developers welcomed BASCOM as an alternative to the popular but slow and clumsy CBASIC. Unlike CBASIC, BASCOM did not need a preprocessor for MBASIC source code so it could be debugged interactively. This gave it a superior edit-compile-run-debug loop compared to CBASIC,

Growing up in the 1970s the BASIC I was first exposed to BASIC on an HP 9830A. It was introduced in 1972 and was the top of the 9800 line, with the

addition of a BASIC interpreter in read-only memory (ROM). HP itself referred to it as a "calculator." This system also had a flatbed plotter that could take an 11x17 sheet of paper. I could write small plotting programs and save them to cassette tape. It also had serial access to the University's PDP-11/40 Running RSTS/E (Resource Sharing Time Sharing / Extended) with BASIC-PLUS.

The high schools in the county all had ASR-33 teletypes with acoustic couplers and could dial into the same PDP-11. This meant I had access to the system during the school year. I managed to get myself an internship at the computer center the summer after my sophomore year of high school. This got me better access to the system and its manuals. I wrote and managed so many BASIC-Plus applications over my college days. Writing system utilities for RSTS/E was a favorite pastime.

Professionally, MBASIC on CP/M and GWBASIC on MS-DOS would be the main BASIC variants until QBasic and Visual Basic for Windows. I am no longer writing BASIC applications in a professional capacity.

Sample Code

The sample program for BASIC is a looped take on the classic Hello World program using CP/M MBASIC and BASCOM.

```
F>MBASIC
BASIC-85 Rev. 5.29
[CP/M Version]
Copyright 1985-1986 (C) by Microsoft
Created: 28-Jul-85
34872 Bytes free
Ok

LOAD "HELLO.BAS"
Ok

LIST
10 FOR I = 1 TO 10
20 PRINT "Hello World! From BASIC line";I
30 NEXT I
40 END
```

```
Ok

RUN
Hello World! From BASIC line 1
Hello World! From BASIC line 2
Hello World! From BASIC line 3
Hello World! From BASIC line 4
Hello World! From BASIC line 5
Hello World! From BASIC line 6
Hello World! From BASIC line 7
Hello World! From BASIC line 8
Hello World! From BASIC line 9
Hello World! From BASIC line 10
Ok

SYSTEM
F>
```

Now, let us use BASCOM to compile the BASIC file into an executable COM file.

```
F>TYPE HELLO.BAS
10 FOR I = 1 TO 10
20 PRINT "Hello World! From BASIC line";I
30 NEXT I
40 END
F>BASCOM =HELLO /E

00000 Fatal Error(s)
24196 Bytes Free

F>L80 HELLO,HELLO/N/E

Link-80  3.44  09-Dec-81  Copyright (c) 1981 Microsoft

Data    4000    4210    <  528>

37990 Bytes Free
[4015   4210       66]

F>HELLO
Hello World! From BASIC line 1
Hello World! From BASIC line 2
Hello World! From BASIC line 3
Hello World! From BASIC line 4
Hello World! From BASIC line 5
Hello World! From BASIC line 6
Hello World! From BASIC line 7
Hello World! From BASIC line 8
```

```
Hello World! From BASIC line 9
Hello World! From BASIC line 10
F>
```

Pascal

Pascal is an imperative and procedural programming language, designed by Niklaus Wirth as a small, efficient language intended to encourage good programming practices using structured programming and data structuring. It is named after French mathematician, philosopher and physicist Blaise Pascal.

Pascal became very successful in the 1970s, notably on the burgeoning minicomputer market. Compilers were also available for many microcomputers as the field emerged in the late 1970s. It was widely used as a teaching language in university-level programming courses in the 1980s, and also used in production settings for writing commercial software during the same period. It was displaced by the C programming language during the late 1980s and early 1990s as UNIX-based systems became popular, and especially with the release of C++.

UCSD Pascal formed the basis of many systems, including Apple Pascal. Borland Pascal was not based on the UCSD codebase, but arrived during the popular period of UCSD and matched many of its features. This started the line that ended with Delphi Pascal and the compatible Open Source compiler FPC/Lazarus.

A derivative named Object Pascal designed for object-oriented programming was developed in 1985. This was used by Apple Computer (for the Lisa and Macintosh machines) and Borland in the late 1980s and later developed into Delphi on the Microsoft Windows platform. Extensions to the Pascal concepts led to the languages Modula-2 and Oberon, both developed by Wirth.

Turbo Pascal became hugely popular, thanks to an aggressive pricing strategy, having one of the first full-screen IDEs, and very fast turnaround time (just seconds to compile, link, and run). It was written and highly optimized entirely in assembly language, making it smaller and faster than much of the competition.

My exposure to Pascal was in college where we had Apple's UCSD Pascal. I started using it professionally on CP/M with Turbo Pascal and then under a Unix distributed multi-processor system to build a medical billing support system for one of the local hospitals. Later I would use it again as Delphi to write Windows Utilities to support the management of media and speech recognition grammars for some educational software development.

Sample Code

The Pascal Hello World program is also a looped version of the Hello World program. We will be using Turbo Pascal for CP/M for this.

```
G>type hello.pas
program hello;
var no : integer;
begin
 ClrScr;
 for no := 1 to 10 do
   writeln('Hello World from Turbo Pascal Line ', no);
end.

G>turb0

G>hello

Hello World from Turbo Pascal Line 1
Hello World from Turbo Pascal Line 2
Hello World from Turbo Pascal Line 3
Hello World from Turbo Pascal Line 4
Hello World from Turbo Pascal Line 5
Hello World from Turbo Pascal Line 6
Hello World from Turbo Pascal Line 7
Hello World from Turbo Pascal Line 8
Hello World from Turbo Pascal Line 9
Hello World from Turbo Pascal Line 10

G>
```

C

C is a general-purpose programming language. It was created in the 1970s by Dennis Ritchie and remains very widely used and influential. By design, C's features cleanly reflect the capabilities of the targeted CPUs. It has found lasting use in operating systems code (especially in kernels), device drivers, and protocol stacks, but its use in application software has been decreasing. C is commonly used on computer architectures that range from the largest supercomputers to the smallest micro-controllers and embedded systems.

C is an imperative procedural language, supporting structured programming, lexical variable scope, and recursion, with a static type system. It was designed to be compiled to provide low-level access to memory and language constructs that map efficiently to machine instructions, all with minimal runtime support. Despite its low-level capabilities, the language was designed to encourage cross-platform programming. A standards-compliant C program written with portability in mind can be compiled for a wide variety of computer platforms and operating systems with few changes to its source code.

It has a static type system. In C, all executable code is contained within subroutines (also called "functions", though not in the sense of functional programming). Function parameters are passed by value, although arrays are passed as pointers, i.e. the address of the first item in the array. Pass-by-reference is simulated in C by explicitly passing pointers to the thing being referenced.

C program source text is free-form code. Semicolons terminate statements, while curly braces are used to group statements into blocks.

I first ran into C in college. While working as a computer lab assistant, I ran across a DECUS tape that had a C compiler that would reportedly run under RSTS/E’s RSX subsystem. I got permission to load the tape and copy off the compiler sources and makefile. The compiler was written in

Macro-11 and while it compiled it did not generate runnable code. A two week journey into the code and I discovered that a piece of the code generator was producing malformed assembler. Correcting the problem got me extra credit in my compiler class.

I would run into C again and again in various Unix systems over the years. Including CP/M, MS-DOS, Windows, Unix and Linux. Even now I am still writing C and C++ applications.

This sample program like the others is also a looped version of the classic Hello World program. We will be using Manx Software System's Aztec C compiler. This compiler is fairly faithful to the original K&R C. Later Hi-Tech C will come along and be more ANSI compliant.

```
E>type hello.c
main()
{
        int i;
        for( i = 1; i <= 10; i++ ) {
            printf("Hello World, from Aztec C line %d\n", i);
        }
}
E>cz hello
C Vers. 1.06D Z80  (C) 1982 1983 1984 by Manx Software Systems

E>as hello
8080 Assembler Vers. 1.06D

D>ln hello.o -lc
C Linker Vers. 1.06D
Base: 0100   Code: 1ff0  Data: 01f8  Udata: 0362  Total: 00254e

E>hello
Hello World, from Aztec C line 1
Hello World, from Aztec C line 2
Hello World, from Aztec C line 3
Hello World, from Aztec C line 4
Hello World, from Aztec C line 5
Hello World, from Aztec C line 6
Hello World, from Aztec C line 7
Hello World, from Aztec C line 8
Hello World, from Aztec C line 9
Hello World, from Aztec C line 10
E>
```

FORTRAN

Fortran, formerly FORTRAN, is a third-generation, compiled, imperative programming language that is especially suited to numeric computation and scientific computing.

The first manual for FORTRAN describes it as a Formula Translating System, and printed the name with small caps, Fortran. Other sources suggest the name stands for Formula Translator or Formula Translation.

Fortran was originally developed by IBM with a reference manual being released in 1956; however, the first compilers only began to produce accurate code two years later. Fortran computer programs have been written to support scientific and engineering applications, such as numerical weather prediction, finite element analysis, computational fluid dynamics, plasma physics, geophysics, computational physics, crystallography and computational chemistry. It is a popular language for high-performance computing[5] and is used for programs that benchmark and rank the world's fastest supercomputers.

Fortran has evolved through numerous versions and dialects. In 1966, the American National Standards Institute (ANSI) developed a standard for Fortran to limit proliferation of compilers using slightly different syntax. Successive versions have added support for a character data type (Fortran 77), structured programming, array programming, modular programming, generic programming (Fortran 90), parallel computing (Fortran 95), object-oriented programming (Fortran 2003), and concurrent programming (Fortran 2008).

Before the development of disk files, text editors and terminals, programs were most often entered on a keypunch keyboard onto 80-column punched cards, one line to a card. The resulting deck of cards would be fed into a card reader to be compiled. Punched card codes included no lower-case letters or many special characters, and special versions of the IBM 026

keypunch were offered that would correctly print the re-purposed special characters used in FORTRAN.

Reflecting punched card input practice, Fortran programs were originally written in a fixed-column format. A letter "C" in column 1 caused the entire card to be treated as a comment and ignored by the compiler. Otherwise, the columns of the card were divided into four fields

Name	Column(s)	Usage
Label Field	1–5	A sequence of digits here was taken as a label for use in DO or control statements such as GO TO and IF, or to identify a FORMAT statement referred to in a WRITE or READ statement. Leading zeros are ignored and 0 is not a valid label number.
Continuation Field	6	A character other than a blank or a zero here caused the card to be taken as a continuation of the statement on the prior card. The continuation cards were usually numbered 1, 2, etc. and the starting card might therefore have zero in its continuation column—which is not a continuation of its preceding card.
Statement Field	7–72	This contains: `DIVISION`, `SECTION` and procedure headers; 01 and 77 level numbers and file/report descriptors
Sequence Field (Ignored)	73–80	Used for identification information, such as punching a sequence number or text, which could be used to re-order cards if a stack of cards was dropped; though in practice this was reserved for stable, production programs.

Within the statement field, whitespace characters (blanks) were ignored outside a text literal. This allowed omitting spaces between tokens for brevity or including spaces within identifiers for clarity. For example, AVG OF X was a valid identifier, equivalent to AVGOFX, and 101010DO101I=1,101 was a valid statement, equivalent to 10101 DO 101 I = 1, 101 because the zero in column 6 is treated as if it were a space (!), while 101010DO101I=1.101 was instead 10101 DO101I = 1.101, the assignment of 1.101 to a variable called DO101I. Note the slight visual difference between a comma and a period.

Hollerith strings, originally allowed only in FORMAT and DATA statements, were prefixed by a character count and the letter H (e.g., 26HTHIS IS ALPHANUMERIC DATA.), allowing blanks to be retained within the character string. Miscounts were a problem.

I mostly ever used Fortran in an educational environment. I had one contract working as a system manager for a PDP-11.34 that ran a set of Fortran programs that were used to model incinerators. I never had to do any code maintenance for that system.

It took me a while to dredge up the memories of how to write a Fortran application. The Microsoft Fortran-80 appears to mostly be ANSI Fortran 66.

Here is another looped version of the Hello World program.

```
H>type hello.for
C Simple Hello World Program
      PROGRAM HelloWorld

      INTEGER  I
      DO 10  I=1, 10
10    WRITE(1, 20)  i
20    FORMAT(33H Hello World! From Fortran, Line , i2)
      END

H>f80 hello,hello=hello
```

```
HELLOW

H>l80 hello,forlib/s,hello/n,/e:hellow

Link-80  3.44  09-Dec-81  Copyright (c) 1981 Microsoft

Data    0103    1A4B    < 6472>

 FORLIB RQUEST
36370 Bytes Free
[012E   1A4B       26]

H>hello

Hello World! From Fortran, Line  1
Hello World! From Fortran, Line  2
Hello World! From Fortran, Line  3
Hello World! From Fortran, Line  4
Hello World! From Fortran, Line  5
Hello World! From Fortran, Line  6
Hello World! From Fortran, Line  7
Hello World! From Fortran, Line  8
Hello World! From Fortran, Line  9
Hello World! From Fortran, Line 10

H>
```

COBOL

COmmon Business-Oriented Language

COBOL is a compiled English-like computer programming language designed for business use. It is an imperative, procedural, and, since 2002, object-oriented language. COBOL is primarily used in business, finance, and administrative systems for companies and governments. COBOL is still widely used in applications deployed on mainframe computers, such as large-scale batch and transaction processing jobs. Many large financial institutions were developing new systems in the language as late as 2006, but most programming in COBOL today is purely to maintain existing applications. Programs are being moved to new platforms, rewritten in modern languages, or replaced with other software.

COBOL statements have prose syntax such as MOVE x TO y, which was designed to be self-documenting and highly readable. However, it is verbose and uses over 300 reserved words compared to the succinct and mathematically inspired syntax of other languages.

The COBOL code is split into four divisions (identification, environment, data, and procedure), containing a rigid hierarchy of sections, paragraphs, and sentences. Lacking a large standard library, the standard specifies 43 statements, 87 functions, and just one class.

The height of COBOL's popularity coincided with the era of keypunch machines and punched cards. The program itself was written onto punched cards, then read in and compiled, and the data fed into the program was sometimes on cards as well.

COBOL can be written in two formats: fixed (the default) or free. In fixed-format, code must be aligned to fit in certain areas (a hold-over from using punched cards). Until COBOL 2002, these were:

Name	Column(s)	Usage
Sequence number area	1–6	Originally used for card/line numbers (facilitating mechanical punched card sorting to assure intended program code sequence after manual editing/handling), this area is ignored by the compiler
Indicator area	7	The following characters are allowed here: • `*` – Comment line • `/` – Comment line that will be printed on a new page of a source listing • `-` – Continuation line, where words or literals from the previous line are continued • `D` – Line enabled in debugging mode, which is otherwise ignored
Area A	8–11	This contains: `DIVISION`, `SECTION` and procedure headers; 01 and 77 level numbers and file/report descriptors
Area B	12–72	Any other code not allowed in Area A
Program name area	73–80	Historically up to column 80 for punched cards, it is used to identify the program or sequence the card belongs to

I used COBOL professionally in several of my jobs. First was R/M COBOL on CP/M. The Service Bureau I worked for sold an accounting package that we did minor customizations for the customers. This is also where I developed the little file clean up utility in assembler.

The next time was at a University IT Department working on an interactive CICS COBOL utilities that allowed employees to request reprints of their W-2 statements. It handled collecting and validating the employee information and managing the print queues.

The last time I had to work with COBOL was the same R/M COBOL Accounting package on SCO Unix on an NCR 68000 Tower.

It took me even longer to remember how to write a COBOL application. The Microsoft COBOL-80 compiler appears to be ANSI COBOL-74 which does not support free formatted code.

Here is yet another looped version of the Hello World program.

```
I>type hello.cob
000100 IDENTIFICATION DIVISION.
000200 PROGRAM-ID. HELLO.
000300 ENVIRONMENT DIVISION.
000400 DATA DIVISION.
000410 WORKING-STORAGE SECTION
000420 01  LINECOUNT PIC 99.
000430 01  DISPCOUNT PIC Z9.
000500 PROCEDURE DIVISION.
000600 00-MAIN.
000700     PERFORM SHOW-MESSAGE VARYING LINECOUNT FROM 1 BY 1
000710         UNTIL LINECOUNT > 10.
000800     STOP RUN.
000900 SHOW-MESSAGE.
000910     MOVE LINECOUNT TO DISPCOUNT.
001000     DISPLAY "Hello World From COBOL, line ", DISPCOUNT.

I>cobol hello,hello=hello/r
Microsoft MS-COBOL
Version 4.65  Copyright 1980,1981,1982 (C) Microsoft

 No Errors or Warnings

I>l80 hello/n,hello/e
Link-80  3.44  09-Dec-81  Copyright (c) 1981 Microsoft

Data    4C00    4F56    <  854>

41731 Bytes Free
[4C44   4F56       79]
```

```
I>hello
Hello World From COBOL, Line  1
Hello World From COBOL, Line  2
Hello World From COBOL, Line  3
Hello World From COBOL, Line  4
Hello World From COBOL, Line  5
Hello World From COBOL, Line  6
Hello World From COBOL, Line  7
Hello World From COBOL, Line  8
Hello World From COBOL, Line  9
Hello World From COBOL, Line 10

I>
```

Modern CP/M Tools

Modern tools related to CP/M primarily involve emulators and utilities for interacting with CP/M systems and files. There are many modern ways to experience CP/M.

- **Modern Hardware** – There are still systems being produced that can run CP/M today.
- **Emulators** – Software emulation of the processor and hardware allows you to run CP/M in your favorite modern environments
- **Utilities** – There are tool that run on Windows, Linux, and MacOS that allow you to manage disk images that are compatible with the emulators and with the proper equipment existing modern and legacy hardware

Modern CP/M Machines

There are several “new” Z-80 machines out there.
Small Computer Central has three varieties:

- Modular backplane and cards. The backplanes come in a few sizes:
    - RCBus 40-pin (standard RC2014 bus)
    - RCBus 80-pin (extended RC2014 bus)
    - Z50Bus (LiNC 50-pin)
- Expandable single board computers (motherboards)
- Non-expandable single board computers (SBC)

A good ROM bios for these systems is RomWBW. This is customizable and can even have a bootable CP/M system disk in the ROM image. The project also has a large selection of CP/M software packaged up as disk images.

I am using the SC-131 the Z-180 Pocket Computer today to run all of the programming examples. They also have systems with the Z-80 processor.

CPUVille,Designing, Building, and Selling Obsolete Computers -- for Educational Purposes -- since 2004 also has a Z-80 computer kit and accessories.

There are even GitHub repositories with new Z-180 systems being designed, such as YAZ180.

Emulators

There are several good software emulators that allow you to run CP/M on modern systems.

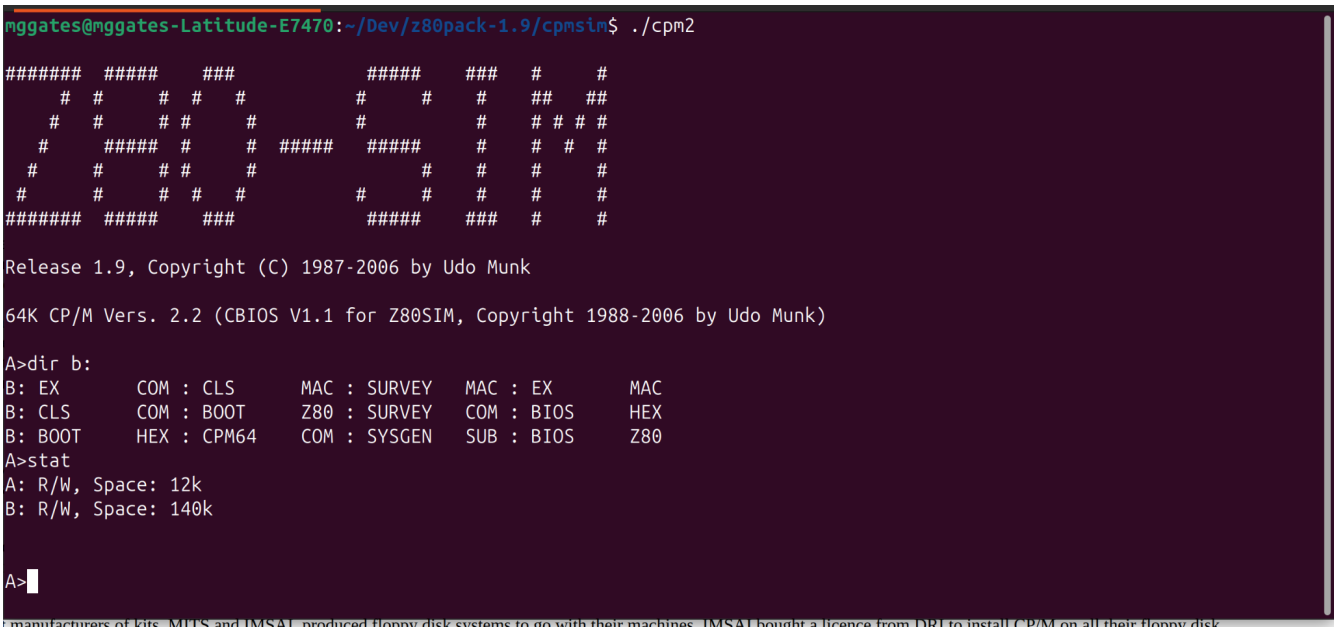
.

These are the ones I worked with for this presentation. They all can be run under Linux, Windows, and MacOS. With the exception of CPMemu which only supports Linux.

Z80Pack

z80pack is a Zilog Z80 and Intel 8080 cross development package for UNIX and Windows systems distributed with all sources under a BSD style license. The CPU emulations are generic and can be used to emulate any Z80 or 8080 based system, the I/O hardware abstraction is well isolated. Originally the software was written for emulation of proprietary Z80 controllers, to support development and testing.

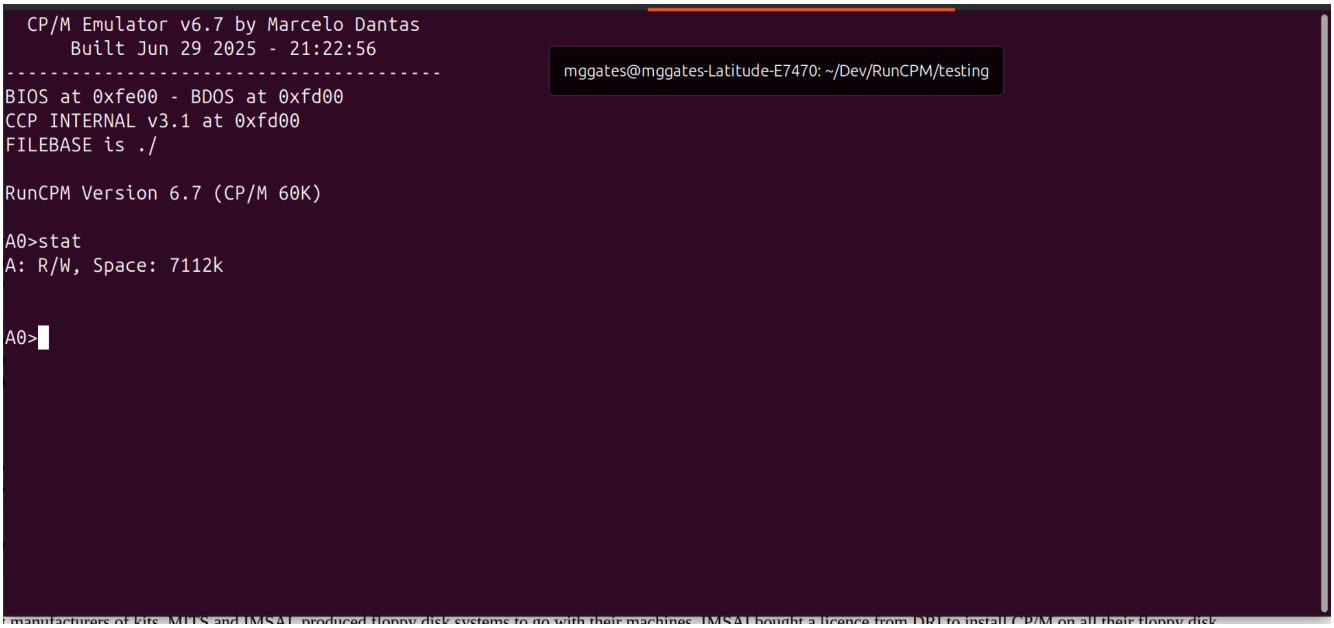
This virtual computer system has then been used to rebuild CP/M 1, CP/M 2, CP/M 3 and MP/M 2 completely from the sources. Also example implementations of CP/NET have been build, to network the virtual systems with this very early implementation of RPC (remote procedure calls). Additional disk images with all sources, tools and build scripts are available for download.



manufacturers of kits. MITS and IMSAI produced floppy disk systems to go with their machines. IMSAI bought a licence from DRI to install CP/M on all their floppy disk

RunCPM

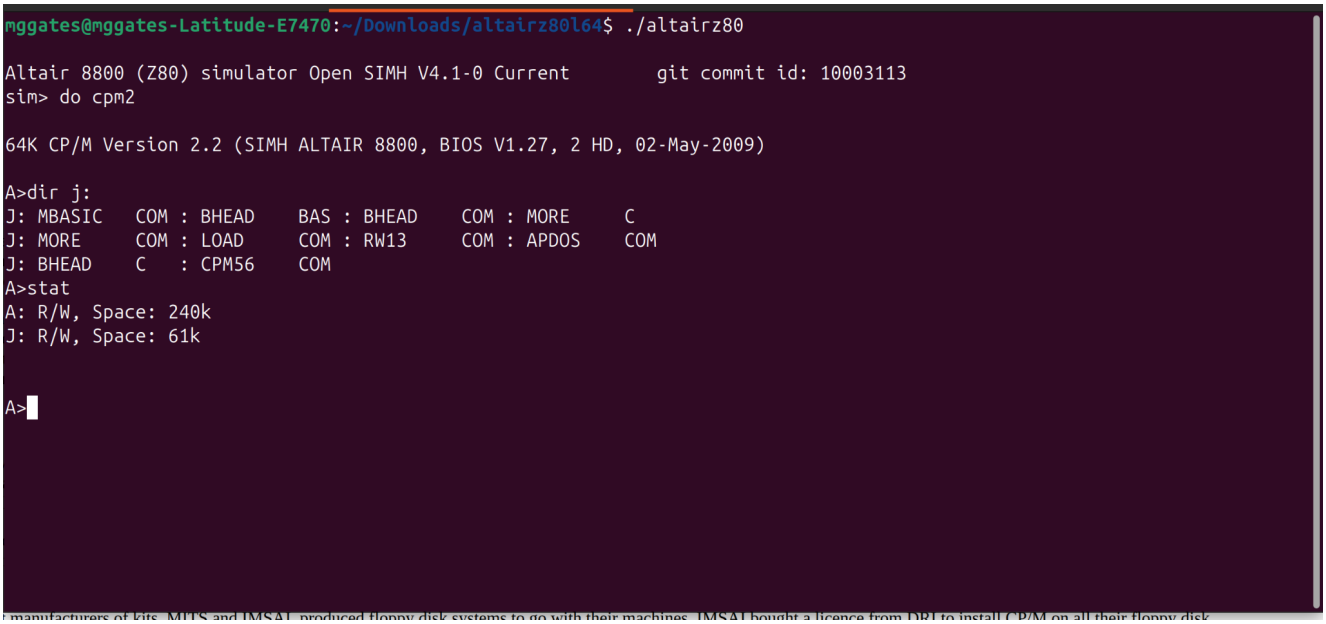
RunCPM is an application that can execute vintage CP/M 8-bit programs on many modern platforms, such as Windows, Mac OS X, Linux, FreeBSD, Arduino DUE, and variants like the Teensy or ESP32. It can be built both on 32 and 64 bits host environments and should be easily portable to other platforms. RunCPM is fully written in C in a modular way, so porting to other platforms should only require writing an abstraction layer file for it. No modification to the main code modules should be necessary.



SIMH

SIMH is a multi-system emulator focused on preserving historic computer systems by simulating their hardware and software. It's a framework and collection of simulators, originally developed by Bob Supnik, that allows users to run software from a variety of old or historical computers on modern systems.

You will need to provide your own disk images to use in the simulation.

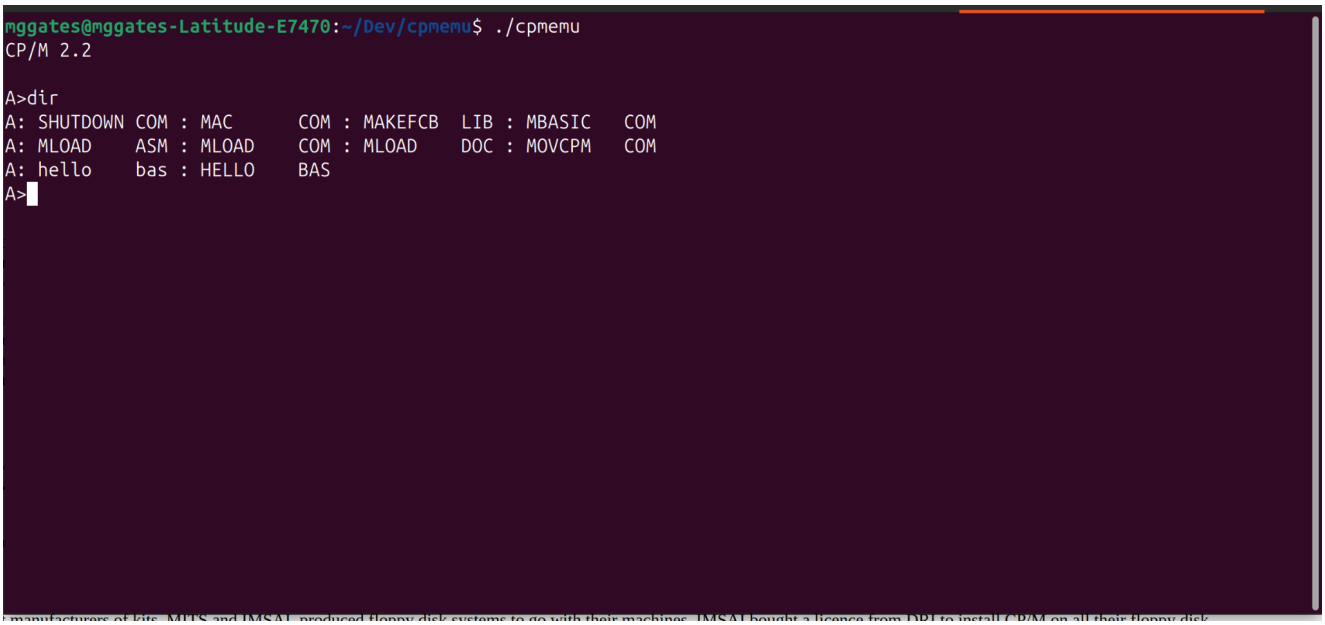


CPMemu

CPMemu is a CP/M 2.2 emulator running on the Raspberry Pi and on other Linux systems. On the raspberry Pi. CPMemu can be used as a bare metal solution based on the Circle environment.

Currently CPMemu does not implement a specific set of terminal control sequences. Instead it sends all characters unchanged to the console.

The installation instructions step you through creating a compatible disk image for use with the emulator.



Utilities

The topic of utilities covers many things: Tools to access disk images and file systems, cross compilers.

cpmtools - Tools to access CP/M file systems.
This package allows you to access CP/M file systems. It can be used for file exchange with a Z80-PC simulator, but it works on floppy devices as well.

Commands available:

- cpmls - list sorted directory with output similar to ls, DIR, P2DOS DIR and CP/M3 DIR
- cpmcp - copy files from and to CP/M file systems
- cpmrm - erase files from CP/M file systems
- cpmchmod - change file permissions
- cpmchattr - change file attributes
- mkfs.cpm - make a CP/M file system
- fsck.cpm - check and repair a CP/M file system
- fsed.cpm - view CP/M file system

ZXCC - CP/M 2/3 emulator for cross-compiling and running CP/M tools.
This is John Elliott's CP/M 2/3 emulator for cross-compiling and running CP/M tools under Microsoft Windows and Linux/Unix/macOS using the latest version of the HI-TECH Z80 C compiler v3.09.

ZXCC is a two-purpose CP/M 2/3 emulator allowing:

- Hi-Tech C for CP/M to be used as a cross-compiler under Unix.
- The CP/M build tools (MAC, RMAC, GENCOM, LINK) to be used under DOS or Unix.

A lot of the functionality of ZXCC is wrapped up in two separate libraries:

- CPMIO (terminal emulation)
- CPMREDIR (filesystem emulation).

The libraries are LGPLed and should be fairly easy to use separately from ZXCC. For example, the stable version of CPMREDIR is also used in the JOYCE PCW emulator.

The main feature of CPMIO is multiple terminal emulations, as seen in MYZ80 under DOS. Current emulations are VT52, VT100 and native (no emulation).

CPMREDIR allows directories on the host system to appear as CP/M drives. ZXCC uses this for all drives; it is also possible to use it to supply only some CP/M drives, as JOYCE does.

Summary

- CP/M was a leap forward for its time.
- As one of the first Commercial Disk Operating Systems, CP/M found its way on to a large number of systems.
- Some of the top business software started on CP/M.
- There were many programming languages developed for CP/M.
- The research for this talk brought back so many memories.

Reference Links

Here are the, currently active, links to all the sites on the web where I gathered information, inspiration and other resources.

Wikipedia Articles

All of the information from WIkipedia is used under the Creative Commons License

Microprocessors

- Microprocessors
- Chronology - 1980s
- Intel 8008
- Intel 8080
- Motorolla 6800
- MOS 6502
- Zilog Z-80

Hardware

- Computers running CP/M
- Intel
- MOS Technology
- Motorola
- SWTPC 6800
- Zilog

CP/M Related

- CP/M
- MP/M
- COM Files

Languages

- Programming Language
- PL/M
- Assembler
- MBASIC
- C Programming Language
- Aztec C
- Turbo Pascal
- COBOL
- Fortran

CM/P Related Sites

- The Humongus CP/M Software Archive
- The Unofficial CP/M Web site
- CP/M Internals
- Gaby's Hompage for CP/M and Computers
- CP/M History, Legacy and Emulation
- Vintage CP/M Computers

Hardware Related Sites

- CPUville - Designing, Building, and Selling Obsolete Computers
- Vintage Computing (deramp.com)
- Small Computer Central (Kits and more)
- Obsolescence Guaranteed
- RomWBW
- Yet Another Z-180

Software Related Sites

- Retrocomputing Archive
- Forth Intrest Group
- Forth-80
- simh references
- ZXCC - C Cross Compiler

Emulator Related Sites

- z80pack
- RunCPM
- open simh
- cpmemu

Various Artilces

- Guidlines for Writting CP/M Software
- 8-BIt History
- Tech Tinkering CP/M Ariticles
- Steve's Old Computer Museum
- Virtue of 8-bit era
- More 8-bit era micros
- retrotechnology.net
- DigiBarn Computer Museum
- Z-80 and CP/M Resources
- Z-80 Official support page
- 50 Years of Personal Computer OS